MONASH University
Information Technology

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the *Copyright Act 1968* (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

MONASH University
Information Technology

FIT5047 – Fundamentals of AI

Knowledge Representation
Chapters 7.1-7.5

Knowledge Representation: Learning Objectives

- **Knowledge-based agents**
- **Logic in general – models and entailment**
- **Propositional (Boolean) logic**
- **Equivalence, validity, satisfiability**
- **First-order logic**
- **Inference rules and theorem proving**
  - forward chaining
  - backward chaining
  - resolution refutation systems
    - > substitution
    - > unification
    - > resolution

Propositional logic

First order logic (next week!)

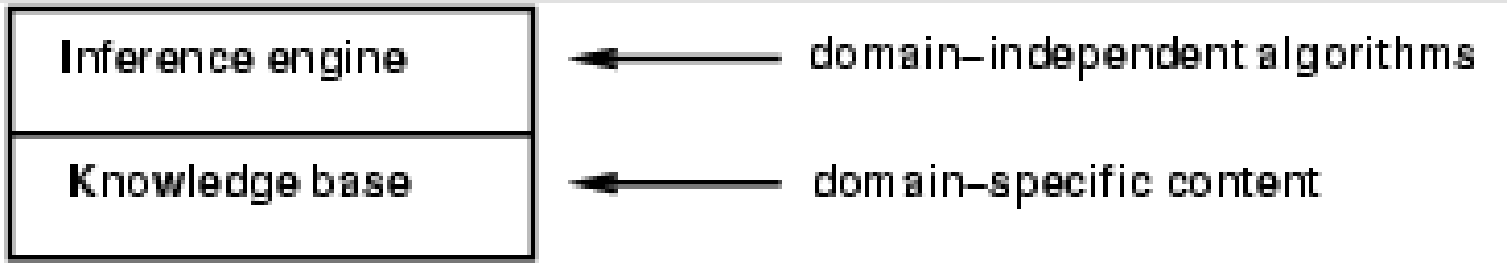
Assumptions about the Environment

- **Observable**
- **Known**
- **Single/multi agent**
- **Deterministic**
- **Sequential/episodic**
- **Static**
- **Discrete**

Knowledge Representation

- **How do we represent facts about the world?**
- **How do we reason about these facts?**
- **Some widely accepted formal calculi**
  - Propositional logic
  - First-order logic
  - Probability calculus

Knowledge Bases



- **Knowledge base = set of sentences in a formal language**
- **Declarative approach to building an agent:**
  - **Tell** it what it needs to know
  - **Ask** it a question – answers follow by inference from the KB

A Simple Knowledge-based Agent

```
function KB-AGENT(percept) returns an action
   static: KB, a knowledge base
           t, a counter, initially 0, indicating time

   TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
   action ← ASK(KB, MAKE-ACTION-QUERY(t))
   TELL(KB, MAKE-ACTION-SENTENCE(action, t))
   t ← t + 1
   return action
```

- **The agent must be able to:**
  - Represent states, actions, etc
  - Incorporate new percepts
  - Update internal representations of the world (KB)
  - Deduce hidden properties of the world
  - Infer appropriate actions

Logic in General

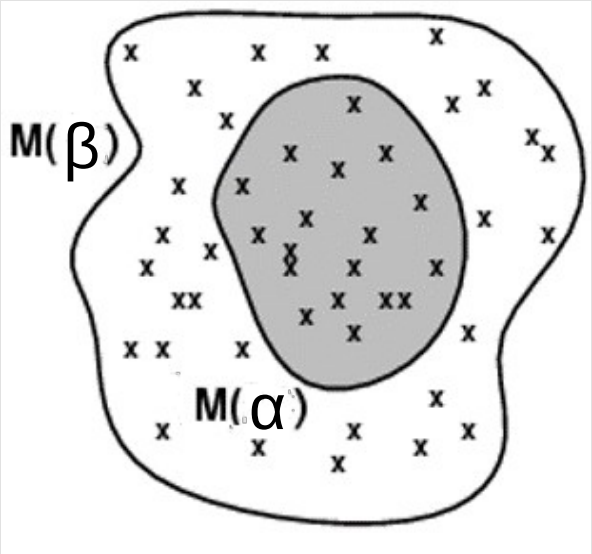
- **Logics are formal languages for representing information so that conclusions can be drawn**
- **Syntax defines what sentences look like**
- **Semantics define the meaning of sentences**
  - the truth of a sentence in each **possible world**
- **E.g., the language of arithmetic**
  - $x+2 \geq y$ is a sentence; $x2+y > \{\}$ is not a sentence
  - $x+2 \geq y$ is true in all the worlds where the number $x+2$ is no less than the number y
    - $x+2 \geq y$ is true in a world where $x = 7$, $y = 1$
    - $x+2 \geq y$ is false in a world where $x = 0$, $y = 6$

Models

- **Models are a formal representation of worlds with respect to which truth can be evaluated**
- **If a sentence α is true in a model *m*, we say that**
  - *m* **is a model of** α, or
  - *m* **satisfies** α

Logical Entailment

- **Entailment means that one sentence follows logically from another: $\alpha \models \beta$**
  $\alpha \models \beta$ iff in every world in which α is true, β is also true.
  The opposite is not necessarily so.
- ***M(α)* = the set of all models of α**
  - $\alpha \models \beta$ iff $M(\alpha) \subseteq M(\beta)$
    - E.g., α = Giants won & Reds won
      β = Giants won



α is the stronger assertion

- **Entailment is a relationship between sentences (i.e., syntax) that is based on semantics**

Inference

- ***KB*** $\vdash_i$ **α means that sentence α can be derived from *KB* by procedure *i***
- **Soundness: procedure *i* is sound if whenever *KB* $\vdash_i$ α, it is also true that *KB* $\models$ α**
- **Completeness: procedure *i* is complete if whenever *KB* $\models$ α, it is also true that *KB* $\vdash_i$ α**
- **First-order logic (FOL) is expressive enough to say almost anything of interest, and there exists a sound and complete inference procedure for it**

Grounding

The connection between logical reasoning processes and real environments

- **How do we know that KB is true in the real world?**
  - The agent's sensors create a connection, i.e., the meaning and truth of percept sentences are defined by the processes of sensing and sentence construction
- **Where do we get the rest of an agent's knowledge?**
  - By learning (generalizing) from experience

MONASH University
Information Technology

FIT5047 – Intelligent Systems

Propositional Logic

Propositional Logic: Some Definitions

- **Literal:** a proposition or its negation
  - E.g., P, ¬P
- **Clause**: a disjunction of literals
  - E.g., ¬P ∨ Q ∨ A

In propositional logic a sentence is a combination of literals related to one another by **logical connectives.**

A clause is one type of sentence

Propositional Logic: Syntax

- **Propositional symbols P_1, P_2, … are <u>atomic</u> sentences**
- **The symbols T(rue) and F(alse) are fixed atoms**
- **Logical connectives (= operators):**
  - **Negation** (¬, read: not):
  - If S is a sentence, ¬S is a sentence
  - **Conjunction** (∧ , read: and):
  - If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence
  - **Disjunction** (∨ , read: or):
  - If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence
  - **Implication** (⇒, read: implies):
  - If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence
  - **Biconditional** (⇔, **read: iff, meaning if and only if):**
  - If S_1 and S_2 are sentences, $S_1 \Leftrightarrow, S_2$ is a sentence
- **Operator precedence:** ¬, ∧ , ∨ , ⇒, ⇔

Propositional Logic: Semantics

- **A model *m* specifies true/false values for each symbol**
  - E.g.,

	S_1	S_2	S_3
	false	true	false

- **Five rules for evaluating any complex sentence with respect to a model *m*:**

$\neg S$	is true iff S is false
$S_1 \wedge S_2$	is true iff S_1 is true and S_2 is true
$S_1 \vee S_2$	is true iff S_1 is true or S_2 is true
$S_1 \Rightarrow S_2$	is true unless S_1 is true and S_2 is false
$S_1 \Leftrightarrow S_2$	is true iff $S_1 \Rightarrow S_2$ is true and $S_2 \Rightarrow S_1$ is true

Truth Tables for Connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
$false$	$false$	$true$	$false$	$false$	$true$	$true$
$false$	$true$	$true$	$false$	$true$	$true$	$false$
$true$	$false$	$false$	$false$	$true$	$false$	$false$
$true$	$true$	$false$	$true$	$true$	$true$	$true$

Propositional Logic: Semantics

- **A model *m* specifies true/false values for each symbol**
  - E.g., S_1 S_2 S_3
    false true false

- **Five rules for evaluating sentences involving logical connectives, with respect to a model *m*:**

$\neg S$	is true iff S is false
$S_1 \wedge S_2$	is true iff S_1 is true and S_2 is true
$S_1 \vee S_2$	is true iff S_1 is true or S_2 is true
$S_1 \Rightarrow S_2$	is true unless S_1 is true and S_2 is false
$S_1 \Leftrightarrow S_2$	is true iff $S_1 \Rightarrow S_2$ is true and $S_2 \Rightarrow S_1$ is true

- **Simple recursive process can evaluate arbitrary sentences**
  - E.g.,
    $\neg S_1 \wedge (S_2 \vee S_3) = \mathit{true} \wedge (\mathit{true} \vee \mathit{false}) = \mathit{true} \wedge \mathit{true} = \mathit{true}$

Propositional Logic: Semantics

Implies is not the same as "therefore"

$S_1 \Rightarrow S_2 \equiv \neg S_1 \vee S_2$ (true iff S_1 is false or S_2 is true)

Think of Implication as a Promise rather than Causality.

Example: You promise you kid you will buy him a lolly if he gets an A on his exam.

A – kid gets an A B – you buy him a lolly

If A is true and B is true, you kept your promise

If A is false and B is true, you kept your promise

If A is false and B is false, you kept your promise.

The only way not to break your promise is:

If A is true and B is false.

Propositional Logic: Semantics

Entails is not the same as "implies"

$\alpha \models \beta \equiv \alpha \Rightarrow \beta$ **<u>in all models</u>**

- Implies talks about truth-value relationships in a particular model
- Entailment talks about truth-value relationships across all models

If we find a model where α is false and β is true, does $\alpha \models \beta$ still hold?
YES (vacuously)

If we find a model where α is false and β is false, does $\alpha \models \beta$ still hold?
YES (vacuously)

If we find a model where α is true and β is true, does $\alpha \models \beta$ still hold?
YES (by definition)

If we find a model where α is true and β is false`, does $\alpha \models \beta$ still hold?
NO (by definition)

Logical Equivalence

- **Two sentences are logically equivalent iff they are both true in the same models:**

- **$\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$**

Logical Equivalence

$(\alpha \wedge \beta)$	$\equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta)$	$\equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma)$	$\equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma)$	$\equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha)$	$\equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta)$	$\equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta)$	$\equiv (\neg\alpha \vee \beta)$	implication elimination
$(\alpha \Leftrightarrow \beta)$	$\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta)$	$\equiv (\neg\alpha \vee \neg\beta)$	de Morgan
$\neg(\alpha \vee \beta)$	$\equiv (\neg\alpha \wedge \neg\beta)$	de Morgan
$(\alpha \wedge (\beta \vee \gamma))$	$\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma))$	$\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

Validity

- **A sentence is <u>valid</u> if it is true in <u>all</u> models**
  - E.g., *True*, $A \vee \neg A$, $A \Rightarrow A$
- **Validity is connected to inference via the Deduction Theorem**
  - $\alpha \models \beta$ iff $\alpha \Rightarrow \beta$ is valid

Validity – Example Proof

Here's a famous valid argument in logic:

- $(A \wedge (A \Rightarrow B)) \Rightarrow B$

- **Let's prove that this argument is valid:**
  - $(A \wedge (\neg A \vee B)) \Rightarrow B$ (equivalence of $\Rightarrow$)
  - $((A \wedge \neg A) \vee (A \wedge B)) \Rightarrow B$ (distribute over $\wedge$)
  - $(A \wedge B) \Rightarrow B$ (simplify)
  - $\neg(A \wedge B) \vee B$ (equivalence of $\Rightarrow$)
  - $\neg A \vee \neg B \vee B$ (distribute over $\vee$)
  - True (tautology)

Satisfiability

- **A sentence is <u>satisfiable</u> if it is true in <u>some</u> model**
  - E.g., $A \vee B, C$
- **A sentence is <u>unsatisfiable</u> if it is true in <u>no</u> model**
  - E.g., $A \wedge \neg A$
- **Satisfiability and validity are connected**
  - α is _____ iff $\neg\alpha$ is unsatisfiable
  - α is _____ iff $\neg\alpha$ is not valid
  - $\alpha \models \beta$ iff $\alpha \wedge \neg\beta$ is _____

Satisfiability

- **A sentence is <u>satisfiable</u> if it is true in <u>some</u> model**
  - E.g., $A \vee B, C$
- **A sentence is <u>unsatisfiable</u> if it is true in <u>no</u> model**
  - E.g., $A \wedge \neg A$
- **Satisfiability and validity are connected**
  - α is valid iff $\neg\alpha$ is unsatisfiable
  - α is satisfiable iff $\neg\alpha$ is not valid
  - $\alpha \models \beta$ iff $\alpha \wedge \neg\beta$ is unsatisfiable

Satisfiability – Example Proofs

- **Prove that α is valid iff ¬α is unsatisfiable**
  - If α is valid, it is true in all models → there does not exist a model for which ¬α is true → ¬α is unsatisfiable
  - If ¬α is unsatisfiable → there does not exist a model for which ¬α is true → α is true in all models → α is valid

- **Prove that α is satisfiable iff ¬α is not valid**
  - If α is satisfiable, it is true in some models → ¬α is not true in these models → ¬α is not valid
  - If ¬α is not valid → there exist some models for which ¬α is false → α is true in these models → α is satisfiable

Proof Methods

- **Model checking**

  Enumerates all possible models to check that a sentence α is true in all models where KB is true

  - **Truth table enumeration** (2^n, where n is the number of symbols)
  - **Improved backtracking** (recursive enumeration of all models) with logic-related heuristics

- **Application of inference rules**

  Sound generation of new sentences from old

  - **Proof** = a sequence of inference rule applications
    - use inference rules as operators in a standard search algorithm
    - typically require transformation of sentences into a ***normal form***

Inference Rules - Modus Ponens

- if "P implies Q" and "P is true", then "Q must also be true“
- A way to eliminate a conditional statement from a logical proof or argument and not carry these antecedents forward in an ever-lengthening string of symbols.
- “If it is raining, then the ground is wet.” and “it is raining” Therefore, “the ground is wet”.

Inference rules	Parent clauses	Resolvent
P and P⇒Q	P and ¬P ν Q	Q

Inference Rules - Modus Tollens

- If a conditional statement ("if P, then Q") is true and the consequent (Q) is false, then the antecedent (P) must also be false.
- The inverse
- “If it’s not raining the ground is not wet” and you know that “The ground is not wet”. Therefore, “it is not raining.”

Inference rules	Parent clauses	Resolvent
$\neg Q$ and $\neg Q \Rightarrow \neg P$	$\neg Q$ and $\neg P \vee Q$	$\neg P$

Common Rules of Inference

Inference rules	Parent clauses	Resolvent	Name
P and P⇒Q	P and ¬P ν Q	Q	Modus Ponens
¬Q and ¬Q ⇒¬P	¬Q and ¬P ν Q	¬P	Modus Tollens
	P and Q	P or Q	And Elimination
¬P ⇒ Q and P⇒Q	P ν Q and ¬P ν Q	Q	
¬Q⇒P and P⇒¬Q	P ν Q and ¬P ν ¬Q	Q ν ¬Q or P ν ¬P	Tautology
	P and ¬P	NIL	
P⇒Q and Q⇒R	¬P ν Q and ¬Q ν R	¬P ν R	Chaining
¬Q⇒P and P⇒R	P ν Q and ¬P ν R	Q ν R	

Rules of Inference – Example

1. Eat ⇒ HaveLessMoney
2. ¬Eat ⇒ Hungry

Rules of Inference – Example

1. Eat ⇒ HaveLessMoney ≡ ¬Eat ∨ HaveLessMoney
2. ¬Eat ⇒ Hungry ≡ Eat ∨ Hungry

Implication:

$\mathbf{S_1 \Rightarrow S_2 \equiv \neg S_1 \vee S_2}$ is true iff S_1 is false or S_2 is true

Rules of Inference – Example

1. Eat ⇒ HaveLessMoney
2. ¬Eat ⇒ Hungry

Eat	HLM	Hungry	¬Eat ∨ HLM	Eat ∨ Hungry	HLM ∨ Hungry
F	F	F	T	F	F
F	F	T	T	T	T
F	T	F	T	F	T
F	T	T	T	T	T
T	F	F	F	T	F
T	F	T	F	T	T
T	T	F	T	T	T
T	T	T	T	T	T

Rules of Inference – Example

1. Eat ⇒ HaveLessMoney
2. ¬Eat ⇒ Hungry

Resolvent:
¬HaveLessMoney ⇒ Hungry

Eat	HLM	Hungry	¬Eat ∨ HLM	Eat ∨ Hungry	HLM ∨ Hungry
F	F	F	T	F	F
F	F	T	T	T	T
F	T	F	T	F	T
F	T	T	T	T	T
T	F	F	F	T	F
T	F	T	F	T	T
T	T	F	T	T	T
T	T	T	T	T	T

Proof as Search

- **Initial state: initial KB**
- **Actions: the inference rules applied to all the sentences that match the LHS of the rule**
- **Result: add the sentence on the RHS of a rule to the KB**
- **Goal: a state where the KB contains the sentence we are trying to prove**

Monotonicity: the set of entailed sentences can only increase as information is added to the KB

Conjunctive Normal Form

- In Boolean logic, **Conjunctive Normal Form** (CNF) is a way to represent a formula as a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals (variables or their negations).

- An expression in CNF is a 'product of sums'.

- The 'sums' are literals (simple propositions or negated propositions, e.g., P , or ¬ Q) **linked by** $\vee$, which are then formed into a 'product' **using** $\wedge$.

Conversion to Conjunctive Normal Form

- **Every sentence in propositional logic is logically equivalent to a conjunction of clauses**
- **Converting a sentence to Conjunctive Normal Form (CNF)**
  1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
  2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$
  3. Move $\neg$ inwards by repeated application of the following equivalences:
     - double-negation: $\neg(\neg\alpha) \equiv \alpha$
     - de Morgan $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$
     - de Morgan $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
  4. Apply distributivity law ($\wedge$ over $\vee$) and flatten

Conversion to CNF – Example

- $A \Leftrightarrow (B \vee C)$

1. Eliminate $\Leftrightarrow$: $(A \Rightarrow (B \vee C)) \wedge ((B \vee C) \Rightarrow A)$
2. Eliminate $\Rightarrow$: $(\neg A \vee (B \vee C)) \wedge (\neg (B \vee C) \vee A)$
3. Move $\neg$ inwards: $(\neg A \vee (B \vee C)) \wedge ((\neg B \wedge \neg C) \vee A)$
   $(\neg A \vee B \vee C) \wedge ((\neg B \wedge \neg C) \vee A)$
4. Apply distributivity law:
   $(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A)$

- Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
- Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$
- Move $\neg$ inwards by repeated application of the following equivalences:
  - double-negation: $\neg (\neg \alpha) \equiv \alpha$
  - de Morgan $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$
  - de Morgan $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
- Apply distributivity law ($\wedge$ over $\vee$) and flatten

The Resolution Inference Rule

- **Resolution – an inference rule applied to clauses that yields a complete inference algorithm when coupled with any complete search algorithm**

parent clauses

$$\frac{l_1 \vee \ldots \vee l_i \vee \ldots \vee l_k \qquad m_1 \vee \ldots \vee m_j \vee \ldots \vee m_n}{l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n}$$

resolvent

  -
  - where l_i and m_j are complementary literals ($l_i = \neg m_j$)
  - the resolvent should contain no copy of each literal
- **Resolution is sound for propositional logic**

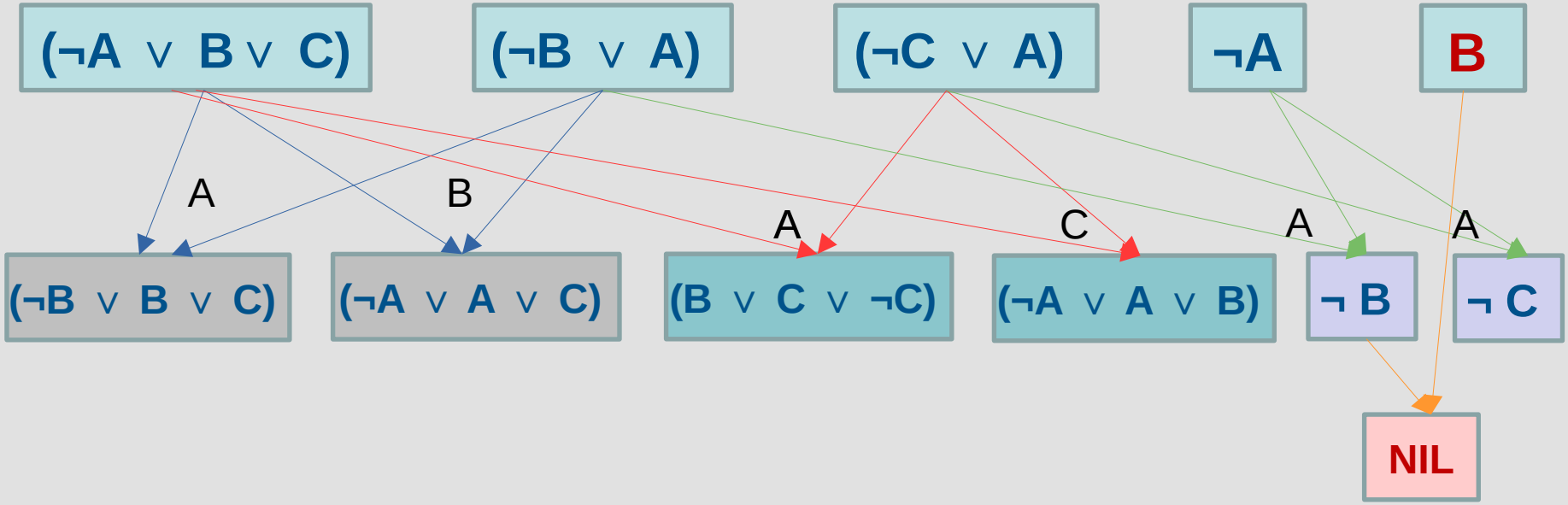
Resolution-refutation Systems

- **Proof by refutation (by negating the original statement)**
  1. Negate the goal and add the negation to the set of clauses
  2. Apply resolution to the clauses in the set of clauses until a contradiction is reached

Resolution-refutation – Example

- ***KB*** **= (A ⇔ (B ∨ C)) ∧ ¬A**

  (¬A ∨ B ∨ C) ∧ (¬B ∨ A) ∧ (¬C ∨ A) ∧ ¬A
- **Prove: α = ¬B**



Resolution-refutation Algorithm

- **Proof by contradiction, i.e., given a goal α, show that *KB* ∧ ¬α is unsatisfiable**

```
function PL-RESOLUTION(KB, α) returns true or false
    clauses ← the set of clauses in the CNF representation of KB ∧ ¬α
    new ← { }
    loop do
        for each Ci, Cj in clauses do
            resolvents ← PL-RESOLVE(Ci, Cj)
            if resolvents contains the empty clause then return true
            new ← new ∪ resolvents
        if new ⊆ clauses then return false
        clauses ← clauses ∪ new
```

Resolution-refutation – Example

KB (rules):

R1: $P \Rightarrow Q$

R2: $L \wedge M \Rightarrow P$

R3: $B \wedge L \Rightarrow M$

R4: $A \wedge P \Rightarrow L$

R5: $A \wedge B \Rightarrow L$

A

B

Prove Q

KB (clauses):

R1: $\neg P \vee Q$

R2: $\neg L \vee \neg M \vee P$

R3: $\neg B \vee \neg L \vee M$

R4: $\neg A \vee \neg P \vee L$

R5: $\neg A \vee \neg B \vee L$

A

B

¬Q

Resolution-refutation:

$\underline{\neg Q}$ R1: $\neg P \vee \underline{Q}$

$\underline{\neg P}$

R2: $\neg L \vee \neg M \vee \underline{P}$ $\underline{\neg P}$

$\neg L \vee \underline{\neg M}$

R3: $\neg B \vee \neg L \vee \underline{M}$ $\neg L \vee \underline{\neg M}$

$\neg B \vee \underline{\neg L}$

R5: $\neg A \vee \neg B \vee \underline{L}$ $\neg B \vee \underline{\neg L}$

$\underline{\neg A} \vee \neg B$

$\underline{A}$ $\underline{\neg A} \vee \neg B$

$\underline{\neg B}$

$\underline{B}$ $\underline{\neg B}$

nil

Horn Clauses and Definite Clauses

- **Definite clause – a disjunction of literals of which <u>exactly one</u> is positive**
  - E.g., $\neg A \lor \neg B \lor C$
  - Definite clauses can be written as implications $(A \land B) \Rightarrow C$
- **Horn clause – a disjunction of literals of which <u>at most one</u> is positive**
  - All definite clauses are Horn clauses
  - Inference with Horn clauses can be done with **forward** or **backward** chaining
  - Deciding entailment with Horn clauses can be done in time that is linear on the size of the KB

Backward Chaining

- **IDEA: Work backwards from the query *q*:**
  - to prove *q* by Backward Chaining, check if *q* is known already, or prove by Backward Chaining all premises of some rule concluding *q*
- **Avoid loops: check if new subgoal is already on the goal stack**
- **Avoid repeated work: check if a new subgoal**
  1. has already been proved true, or
  2. has already failed

Backward chaining is sound and complete for Horn KBs

Backward Chaining – Example

- **KB:**

  R1: $P \Rightarrow Q$

  R2: $L \wedge M \Rightarrow P$

  R3: $B \wedge L \Rightarrow M$

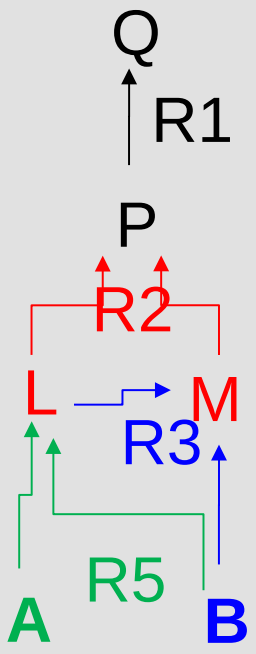
  R4: $A \wedge P \Rightarrow L$

  R5: $A \wedge B \Rightarrow L$

  A

  B

- **Prove Q**



Forward Chaining Algorithm

- **IDEA: Work forwards from the facts in KB**

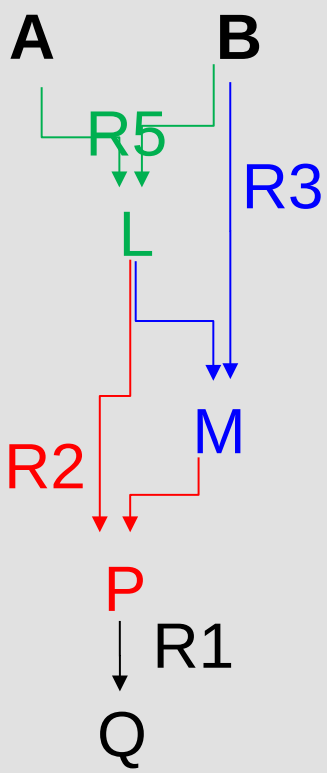
```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
          q, the query, a proposition symbol
  count ← a table, where count[c] is initially the number of symbols in clause c's premise
  inferred ← a table, where inferred[s] is initially false for all symbols
  queue ← a queue of symbols, initially symbols known to be true in KB

  while queue is not empty do
     p ← POP(queue)
     if p = q then return true
     if inferred[p] = false then
        inferred[p] ← true
        for each clause c in KB where p is in c.PREMISE do
           decrement count[c]
           if count[c] = 0 then add c.CONCLUSION to queue
  return false
```

Forward chaining is sound and complete for Horn KBs

Forward Chaining – Example

- **KB:**
  - R1: $P \Rightarrow Q$
  - R2: $L \wedge M \Rightarrow P$
  - R3: $B \wedge L \Rightarrow M$
  - R4: $A \wedge P \Rightarrow L$
  - R5: $A \wedge B \Rightarrow L$
  - A
  - B
- **Prove Q**



Queue	Count	Inferred
	R1 R2 R3 R4 R5	
A B	1 2 2 2 2	

Forward versus Backward Chaining

- **Forward Chaining**
  - **data-driven**, automatic, unconscious processing
    - e.g., object recognition, routine decisions
  - may do lots of work that is irrelevant to the goal
- **Backward Chaining**
  - **goal-driven**, appropriate for problem-solving
    - e.g., How do I get into a PhD program?
  - complexity can be much less than linear in size of KB
- **Both are sound and complete for Horn KBs**

Pros and Cons of Propositional Logic

- ☺ **declarative**
- ☺ **allows partial/disjunctive/negated information**
- ☺ **compositional:**
  - meaning of $A \wedge B$ is derived from meaning of A and of B
- ☺ **Meaning is context-independent**

Propositional logic has very limited expressive power

- E.g., cannot say “all men are mortal“